COMP 3522

OBJECT ORIENTED PROGRAMMING 2

Midterm Exam

October 2024

Full Name	
Signature	
Student ID	
Set	

Instructions:

1. No phones. Turn off your phone and put it away. If your phone rings you will earn zero on this exam.
2. This is a closed-book exam. Two cheat sheet allowed. You may not use any notes, texts, manuals, etc., during the exam
3. Headphones are OK.
4. This midterm is 90 minutes long.
5. No candidate will be admitted after 30 minutes.
6. No candidate will be permitted to leave during the first 30 minutes.
7. Add The answers for T/F and MCQ questions to the Answer sheet in Page 2.
8. Good luck. You're awesome, you've got this.

Final grade:

Name		Date	
Class		Quiz	

Student ZipGrade ID

Key Version
A
B
C
D
E

		A	B	C	D	E		A	B	C	D	E		A	B	C	D	E
1		○	○	○	○	○	11	○	○	○	○	○	31	○	○	○	○	○
2		○	○	○	○	○	12	○	○	○	○	○	32	○	○	○	○	○
3		○	○	○	○	○	13	○	○	○	○	○	33	○	○	○	○	○
4		○	○	○	○	○	14	○	○	○	○	○	34	○	○	○	○	○
5		○	○	○	○	○	15	○	○	○	○	○	35	○	○	○	○	○
6		○	○	○	○	○	16	○	○	○	○	○	36	○	○	○	○	○
7		○	○	○	○	○	17	○	○	○	○	○	37	○	○	○	○	○
8		○	○	○	○	○	18	○	○	○	○	○	38	○	○	○	○	○
9		○	○	○	○	○	19	○	○	○	○	○	39	○	○	○	○	○
0		○	○	○	○	○	20	○	○	○	○	○	40	○	○	○	○	○
	A	B	C	D	E		A	B	C	D	E		A	B	C	D	E	
1	○	○	○	○	○	21	○	○	○	○	○	41	○	○	○	○	○	
2	○	○	○	○	○	22	○	○	○	○	○	42	○	○	○	○	○	
3	○	○	○	○	○	23	○	○	○	○	○	43	○	○	○	○	○	
4	○	○	○	○	○	24	○	○	○	○	○	44	○	○	○	○	○	
5	○	○	○	○	○	25	○	○	○	○	○	45	○	○	○	○	○	
6	○	○	○	○	○	26	○	○	○	○	○	46	○	○	○	○	○	
7	○	○	○	○	○	27	○	○	○	○	○	47	○	○	○	○	○	
8	○	○	○	○	○	28	○	○	○	○	○	48	○	○	○	○	○	
9	○	○	○	○	○	29	○	○	○	○	○	49	○	○	○	○	○	
10	○	○	○	○	○	30	○	○	○	○	○	50	○	○	○	○	○	

Students: Fill circles completely with black ink or pencil.
Erase all stray marks completely.

Teachers: Hold paper on flat surface when grading.
Be aware of bright lights and shadows.

ZIPGRADE.COM

True/False Questions

[4 Marks]

Question 1

The following code demonstrate how you can **unpack** key-value pairs from dictionaries using **.

```
my_dict = {'x': 10, 'y': 20}
def example(x, y):
    print(x, y)
example(**my_dict)
# x = 10, y = 20
```

Question 2

The following code demonstrate **packing** in python.

```
a, *b = [1, 2, 3, 4]
# a = 1, b = [2, 3, 4]
```

Question 3

Given the following code:

```
from datetime import datetime, timedelta

current_date = datetime(2024, 10, 20)
future_date = current_date +
timedelta(days=10, hours=5)

result = future_date - current_date
```

The value of `result.days` will be `10`.

Question 4

The Interface Segregation Principle (ISP) advocates that classes should implement large, general-purpose interfaces to minimize the number of interfaces in a system.

Question 5

The Dependency Inversion Principle (DIP) recommends that high-level modules should depend on low-level modules, ensuring that the high-level modules can directly control the low-level functionality.

Question 6

When using the `range()` function in Python, the default step value is `0` if you do not explicitly specify it.

Question 7

```
my_list = [10, 20, 30, 40, 50]
result = my_list[3:0:-1]
```

The value of `result` will be `[40, 30, 20]`.

Question 8

```
r = range(-5, 5, 2)
result = list(r)
```

The value of `result` will be `[-5, -3, -1, 1, 3]`.

Question 9

When opening a file in `"w"` mode, if the file already exists, Python will raise an `IOError` because it does not allow overwriting existing files by default.

Question 10

In `cProfile` output, the *cumulative time* for a function includes the time spent in the function itself, as well as the time spent in all the functions it calls, including recursive calls.

MCQ Questions

[10 Marks]

Question 1

Match the method of multi-line string handling with the best description and use case:

Method	Typical Use Case
1. Backslash (\)	i. Multi-line strings, preserving line breaks
2. Triple Quotes (''' or """)	ii. Breaking up long strings with no line breaks
3. Parentheses (())	iii. Breaking up code or strings for readability

---

Options:

A)

```
1, iii
2, i
3, ii
```

B)

```
1, ii
2, i
3, iii
```

C)

```
1, i
2, ii
3, iii
```

D)

```
1, iii
2, ii
3, i
```

Question 2

Given the following code, what will the output be?

```
my_list = [10, 20, 30, 40]
print(my_list[1:3] + my_list[-2:])
```

A) `[20, 30, 40, 50]`
B) `[20, 30, 30]`
C) `[20, 30, 40]`
D) `[20, 30, 30, 40]`

Question 3

What is the purpose of name mangling, and how does it work in Python?

A) To enforce true private variables, preventing any access from outside the class.
B) To rename variables with a single underscore to indicate internal use, but allow access from anywhere.
C) To change the variable name by prefixing it with the class name to avoid accidental access or overriding in subclasses.
D) To make class methods callable only within the class by adding an underscore prefix to the function name.

Question 4

Consider the following code snippet:

```
def update_value(n):
    n += 5
    return n

x = 10
y = update_value(x)
print(x, y)
```

What will be the output of the above code?

A) `10 15`
B) `15 10`
C) `10 10`
D) `15 15`

Question 5

In a UML class diagram, which of the following best describes the relationship between `ClassA` and `ClassB` if `ClassA` has an instance variable of type `ClassB` that is created outside of `ClassA`?

A) **Association** - `ClassA` contains `ClassB` as an attribute, but `ClassB` is created within `ClassA`.
B) **Aggregation** - `ClassA` and `ClassB` are related, where `ClassB` is passed into `ClassA`, and its existence is independent of `ClassA`.
C) **Composition** - `ClassA` and `ClassB` are tightly coupled; if `ClassA` is destroyed, `ClassB` will also be destroyed.
D) **Dependency** - `ClassA` uses a method from `ClassB`, but `ClassB` is not stored as an instance variable.

Question 6

Given the following function definition, which call to the function is **invalid**?

```
def process_data(a, b=5, *args, **kwargs):
    print(a, b, args, kwargs)
```

A) `process_data(1, 2, 3, 4, x=10, y=20)`

B) `process_data(1, 2, x=10, y=20)`

C) `process_data(1, x=10, y=20, b=2)`

D) `process_data(x=10, y=20)`

Questions 7, 8, & 9

Use the following code to answer the following three questions. The code contains a simple dictionary program that loads words and definitions from a JSON or TXT file, and allows users to query definitions for specific words. The program uses a `Dictionary` class to manage the dictionary data and a `FileHandler` class to load and write data to files.

```
import difflib
import unittest
from unittest.mock import mock_open, patch

import json
from pathlib import Path
from enum import Enum


# Enum to handle file types
class FileExtensions(Enum):
    TXT = "txt"
    JSON = "json"


# Custom exception for invalid file types
class InvalidFileTypeError(Exception):
    pass


class FileHandler:

    @staticmethod
    def load_data(path, file_extension_enum):
        file_path = Path(path)

        if file_extension_enum == FileExtensions.JSON:
            with open(file_path, "r") as file:
                data = json.load(file)
        elif file_extension_enum == FileExtensions.TXT:
            with open(file_path, "r") as file:
                data = {}
                for line in file:
                    word, definition =
line.strip().split(":")
                    data[word.strip()] =
definition.strip()
        else:
            raise InvalidFileTypeError(
                f"Unsupported file type: {

file_extension_enum}"
            )

        return data

    @staticmethod
    def write_lines(path, lines):
        with open(path, "a") as file:
            for line in lines:
                file.write(line + "\n")
```

```
class Dictionary:

    def __init__(self):
        self.dictionary = {}
        self.loaded = False

    def load_dictionary(self, filepath):
        file_extension = filepath.split(".")[-1]

        if file_extension == FileExtensions.JSON.value:
            self.dictionary =
FileHandler.load_data(filepath, FileExtensions.JSON)
        elif file_extension == FileExtensions.TXT.value:
            self.dictionary =
FileHandler.load_data(filepath, FileExtensions.TXT)
        else:
            raise InvalidFileTypeError("Only .json and
.txt files are supported")

        self.loaded = True

    def query_definition(self, word):
        word = word.lower()
        if word in self.dictionary:
            return self.dictionary[word]
        else:
            suggestions =
difflib.get_close_matches(word, self.dictionary.keys())
            if suggestions:
                return f"Did you mean: {',
'.join(suggestions)}?"
            else:
                return "Word not found in dictionary."


class TestDictionaryProgram(unittest.TestCase):

    def setUp(self):
        # Set up a mock dictionary object before each
test
        self.dictionary = Dictionary()

    def test_query_definition(self):
        # TODO: Implement this test

    def test_query_word_not_found_with_suggestion(self):
        # TODO: Implement this test

    def test_load_invalid_file_extension(self):
        # TODO: Implement this test


# Entry point for running the tests
if __name__ == "__main__":
    unittest.main()
```

Question 7

Which of the following is a proper test for the `query_definition` method of the `Dictionary` class?

Options:

A)

```
    def test_query_definition(self):
        self.dictionary.dictionary = {"rain": "precipitation", "sun": "sunlight"}
        result = self.dictionary.query_definition("rain")
        self.assertEqual(result, "precipitation")
```

B)

```
    def test_query_definition(self):
        self.dictionary.dictionary = {"rain": "precipitation", "sun": "sunlight"}
        result = self.dictionary.query_definition("rain")
        self.assertEqual(result, "sunlight")
```

C)

```
    def test_query_definition(self):
        result = self.dictionary.query_definition("rain")
        self.assertEqual(result, "precipitation")
```

D)

```
    def test_query_definition(self):
        self.dictionary.dictionary = {"rain": "precipitation", "sun": "sunlight"}
        result = self.dictionary.get_definition("rain")
        self.assertEqual(result, "precipitation")
```

Question 8

What is the proper way to test the query word not found with suggestion method of the Dictionary class?

A)

```
    def test_query_word_not_found_with_suggestion(self):
        self.dictionary.dictionary = {"rain": "precipitation", "sun": "sunlight"}
        result = self.dictionary.query_definition("ran")
        self.assertIn("Did you mean: rain?", result)
```

B)

```
    def test_query_word_not_found_with_suggestion(self):
        self.dictionary.dictionary = {"rain": "precipitation", "sun": "sunlight"}
        result = self.dictionary.query_definition("ran")
        self.assertEqual(result, "Did you mean: rain?")
```

C)

```
    def test_query_word_not_found_with_suggestion(self):
        self.dictionary.dictionary = {}
        result = self.dictionary.query_definition("ran")
        self.assertIn("Did you mean: rain?", result)
```

D)

```
    def test_query_word_not_found_with_suggestion(self):
        self.dictionary.dictionary = {"sun": "sunlight"}
        result = self.dictionary.query_definition("ran")
        self.assertIn("Did you mean: sun?", result)
```

Question 9

Which of the following options is equivalent to the following test case:

```
    def test_load_invalid_file_extension(self):
        with self.assertRaises(InvalidFileTypeError):
            self.dictionary.load_dictionary("data.pdf")
```

Options:

A)

```
    def test_load_invalid_file_extension(self):
        try:
            self.dictionary.load_dictionary("data.pdf")
        except InvalidFileTypeError:
            self.fail("This exception should not have been raised.")
```

B)

```
    def test_load_invalid_file_extension(self):
        try:
            self.dictionary.load_dictionary("data.pdf")
            self.fail("Expected InvalidFileTypeError but no exception was raised.")
        except FileNotFoundError:
            pass
```

C)

```
    def test_load_invalid_file_extension(self):
        try:
            self.dictionary.load_dictionary("data.pdf")
        except InvalidFileTypeError:
            pass
        self.fail("Expected InvalidFileTypeError but no exception was raised.")
```

D)

```
    def test_load_invalid_file_extension(self):
        try:
            self.dictionary.load_dictionary("data.pdf")
            self.fail("Expected InvalidFileTypeError but no exception was raised.")
        except InvalidFileTypeError:
            pass
```

Question 10

What will be the output of the following code snippet?

```
from enum import Enum

class Status(Enum):
    SUCCESS = 1
    FAILURE = 2
    ERROR = 2

print(Status.ERROR == Status.FAILURE)
```

A) `True`
B) `False`
C) `TypeError: Cannot compare enum members`
D) `AttributeError: 'Status' object has no attribute 'ERROR'`

Short Answers Questions

Question 1 – Code Completion

[2 Marks]

Complete the class `PrimeNumberIterator`, which is designed to iterate over prime numbers starting from 2, up to a specified number of terms. You need to implement the `__next__` method to return the next prime number in the sequence.

Complete the `__next__` method so that it works correctly.

```
class PrimeNumberIterator:
    """
    An iterator that iterates over prime numbers.
    """

    def __init__(self, num_terms):
        """
        Initializes the iterator with the number of primes to return.
        :param num_terms: number of prime numbers to return
        """
        self.num_terms = num_terms
        self.count = 0  # Keeps track of how many primes have been returned
        self.current = 2  # The first prime number

    def __iter__(self):
        """
        Returns the iterator object itself.
        :return: PrimeNumberIterator
        """
        return self

    def is_prime(self, n):
        """
        Helper method to determine if a number is prime.
        :param n: the number to check
        :return: True if n is prime, False otherwise
        """
        if n < 2:
            return False
        for i in range(2, int(n ** 0.5) + 1):
            if n % i == 0:
                return False
        return True

    def __next__(self):
        # Your code here
```

Example Usage

```
iterator = PrimeNumberIterator(5)
for prime in iterator:
    print(prime)
```

Expected Output:

```
2
3
5
7
11
```

Your Solution:

```
def __next__(self):
    if self.count == self.num_terms:
        raise StopIteration
    while True:
        if self.is_prime(self.current):
            prime_number = self.current
            self.current += 1
            self.count += 1
            return prime_number
        self.current += 1
```

Question 2 – Write Output

[1 Mark]

What are the MROs of the following class structure. Show all intermediate steps.

Code Example

```
class X:
    def method(self):
        print("Method from X")

class Y(X):
    def method(self):
        print("Method from Y")

class Z(X):
    def method(self):
        print("Method from Z")

class A(Y):
    def method(self):
        print("Method from A")

class B(Z):
    def method(self):
        print("Method from B")

class C(A, B):
    pass
```

Your Solution:

L(C) = [C] + m(L(A), L(B), AB)

= C + m(AYX, BZX, AB)

= CA + m(YX, BZX, B)

= CAY + m(X, BZX, B)

= CAYB + m(X, ZX)

= CAYBZX

Long Answer Question - SOLID Principles

[4 Marks]

The following questions are based on the SOLID principles of object-oriented design. Read the following code example and answer the questions that follow.

```python
# Context Class (Enemy) without SOLID principles
class Enemy:
    def __init__(self, name):
        self.name = name
        self.movement_type = "aggressive"  # Default movement type

    def set_movement_type(self, movement_type):
        self.movement_type = movement_type

    def perform_move(self, player):
        if self.movement_type == "aggressive":
            self.move_aggressively(player)
        elif self.movement_type == "defensive":
            self.move_defensively(player)
        elif self.movement_type == "evasive":
            self.move_evasively(player)
        else:
            print(f"{self.name} doesn't know how to move.")

    def move_aggressively(self, player):
        print(f"{self.name} is moving aggressively toward {player.name}")

    def move_defensively(self, player):
        print(f"{self.name} is moving defensively, keeping distance from {player.name}")

    def move_evasively(self, player):
        print(f"{self.name} is moving evasively, avoiding {player.name}")

    def update(self, player):
        # Logic for updating movement type based on player's state
        if player.is_near():
            self.set_movement_type("aggressive")
        elif player.is_far():
            self.set_movement_type("defensive")
        else:
            self.set_movement_type("evasive")

# Example usage
player = Player("Hero")  # Assume a Player class exists
enemy = Enemy("Orc")  # Starts with aggressive movement by default

enemy.perform_move(player)  # Output: Orc is moving aggressively toward Hero

# Change movement type to defensive at runtime
enemy.set_movement_type("defensive")
enemy.perform_move(player)  # Output: Orc is moving defensively, keeping distance from Hero
```

Question 1

In few words, what is the main issue with the previous implementation of the Enemy class?

The class is too big. It handles movements which can be abstracted in its own class

Question 2

Modify the previous code to add support for *Stealth Movement*; In the `update()` method of the Enemy class, add a new condition to set the movement type to “stealth” when the player is searching for the enemy. Also, update the `perform_move` method to handle the new “stealth” movement type.

Boxes are added to guide where to write code.

```
class Enemy:
    def __init__(self, name):
        self.name = name
        self.movement_type = "aggressive"  # Default movement type

    def set_movement_type(self, movement_type):
        self.movement_type = movement_type

    def perform_move(self, player):
        if self.movement_type == "aggressive":
            self.move_aggressively(player)
        elif self.movement_type == "defensive":
            self.move_defensively(player)
        elif self.movement_type == "evasive":
            self.move_evasively(player)
        elif self.movement_type == "stealth":
            self.move_stealthily(player)
        else:
            print(f"{self.name} doesn't know how to move.")

    def move_aggressively(self, player):
        print(f"{self.name} is moving aggressively toward {player.name}")

    def move_defensively(self, player):
        print(f"{self.name} is moving defensively, keeping distance from {player.name}")

    def move_evasively(self, player):
        print(f"{self.name} is moving evasively, avoiding {player.name}")

    def move_stealthily(self, player):
        print(f"{self.name} is moving stealthily, staying hidden from {player.name}")  # New method for stealth movement

    def update(self, player):
        # Logic for updating movement type based on player's state
        if player.is_near():
            self.set_movement_type("aggressive")
        elif player.is_far():
            self.set_movement_type("defensive")
        elif player.is_searching():
            self.set_movement_type("stealth")
        else:
            self.set_movement_type("evasive")

# Example usage
player = Player("Hero")  # Assume a Player class exists
enemy = Enemy("Orc")  # Starts with aggressive movement by default

enemy.perform_move(player)  # Output: Orc is moving aggressively toward Hero

# Change movement type to defensive at runtime
enemy.set_movement_type("defensive")
enemy.perform_move(player)  # Output: Orc is moving defensively, keeping distance from Hero
```

Question 3

Using SOLID principles, let us refactor the Enemy class to improve its design. Introduce the abstraction of a MovementStrategy interface and implement concrete movement strategies for aggressive, defensive, and evasive movements.

Write code in white space. You may also use labels or numbers instead. A snippet can be used more than once.

```
class MovementStrategy:
    def move(self, enemy, player):
        pass

class AggressiveMovement(MovementStrategy):
    def move(self, enemy, player):
        # Logic to move towards the player
        print(f"{enemy.name} is moving
aggressively toward {player.name}")

class DefensiveMovement(MovementStrategy):
    def move(self, enemy, player):
        # Logic to maintain a safe distance
        print(f"{enemy.name} is moving
defensively, keeping distance from
{player.name}")

class EvasiveMovement(MovementStrategy):
    def move(self, enemy, player):
        # Logic to evade the player
        print(f"{enemy.name} is moving evasively,
avoiding {player.name}")
```

```
class Enemy:
    def __init__(self, name, new_strategy:
MovementStrategy):
        self.name = name
        self.movement_strategy = new_strategy

    def set_movement_strategy(self, new_strategy:
MovementStrategy):
        self.movement_strategy = new_strategy

    def perform_move(self, player):
        self.movement_strategy.move(self, player
```

```
class EvasiveMovement(MovementStrategy):
```

```
print(f"{enemy.name} is moving defensively, keeping
distance from {player.name}")
```

```
    def move(self, enemy, player):
        pass
```

```
print(f"{enemy.name} is moving aggressively toward
{player.name}")
```

```
def move(self, enemy, player):
```

```
class DefensiveMovement(MovementStrategy):
```

```
        self.movement_strategy.move(self, player)
```

```
        self.movement_strategy = new_strategy
```

```
class AggressiveMovement(MovementStrategy):
```

```
        self.name = name
```

```
        print(f"{enemy.name} is moving evasively,
avoiding {player.name}")
```

```
# Usage Example
player = Player("Hero")  # Assume a Player class exists
enemy = Enemy("Orc", AggressiveMovement())  # Starts with aggressive movement

enemy.perform_move(player)  # Output: Orc is moving aggressively toward Hero

# Change strategy to defensive at runtime
enemy.set_movement_strategy(DefensiveMovement())
enemy.perform_move(player)  # Output: Orc is moving defensively, keeping distance from Hero
```

Question 4

Which SOLID principle is being applied in the refactored Enemy class? Breifly, explain how the refactored design adheres to this principle.

DIP: Enemy class (high-level module) should not depend on movement (low-level modules) which increases flexibility and reduces coupling.

THE END 🙂